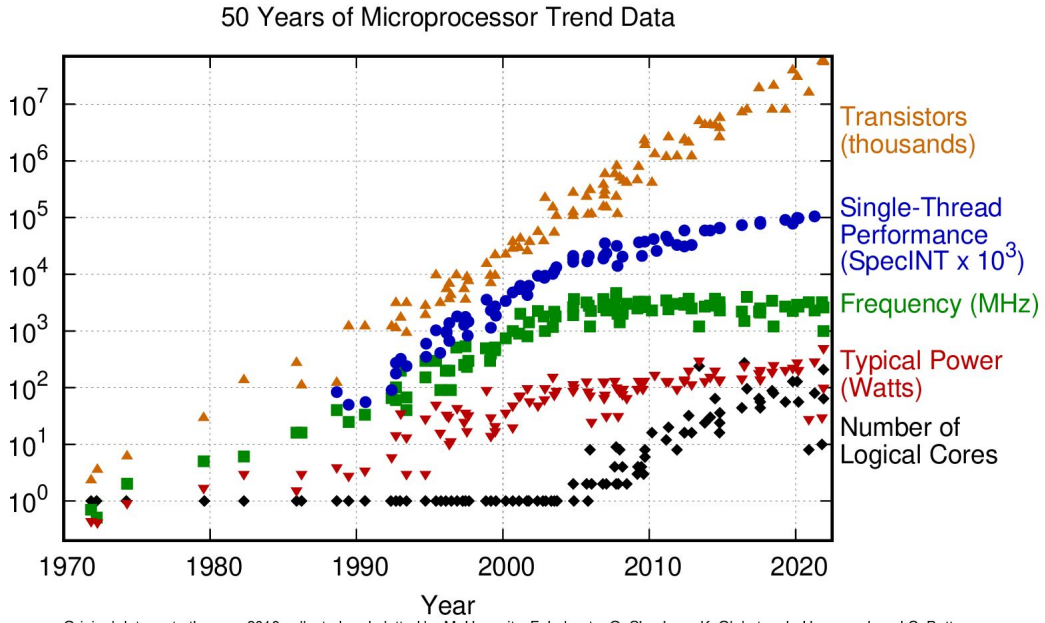
GPUs: Concurrency at a Massive Scale

Samantha Dreussi
University of Washington

Beyond the CPU

- Single-core CPU performance has essentially stagnated
- 20 years ago, solution was multicore and increased parallelism
- Programs need increased performance

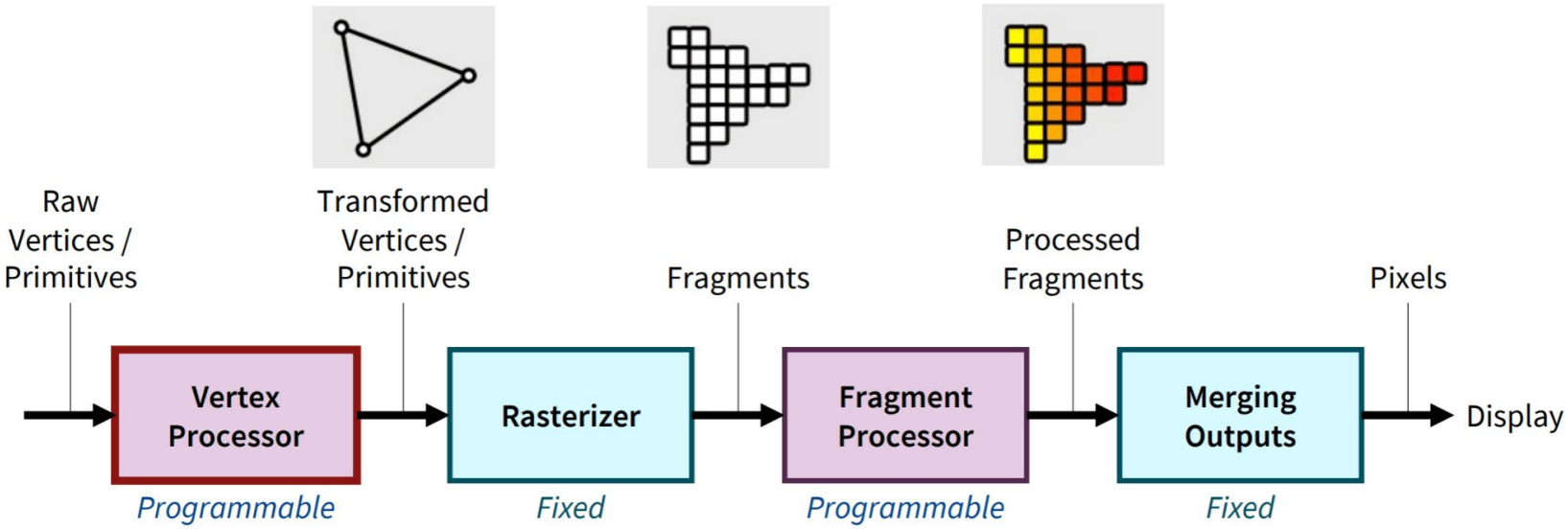


Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

Enter the GPU

[4]

History & Evolution



[3]

How did we get here?

Linear Algebra Operators for GPU Implementation of Numerical Algorithms

Jens Krüger and Rüdiger Westermann
Computer Graphics and Visualization Group, Technical University Munich*

Brook for GPUs: Stream Computing on Graphics Hardware

Ian Buck Tim Foley Daniel Horn Jeremy Sugerman Kayvon Fatahalian Mike Houston Pat Hanrahan
Stanford University

ImageNet Classification with Deep Convolutional Neural Networks

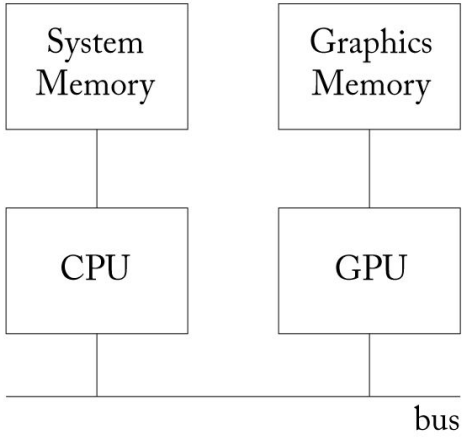
Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

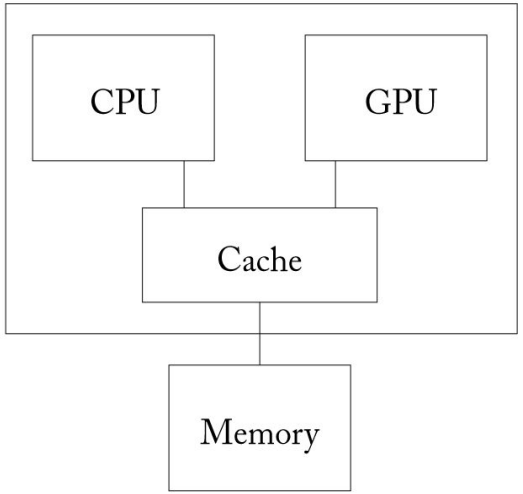
Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Architecture Overview

- CPU initiates computation on the GPU
  - Launches a “kernel”
- Discrete GPUs often have their own dedicated memory
- Integrated GPUs often share w/ host CPU



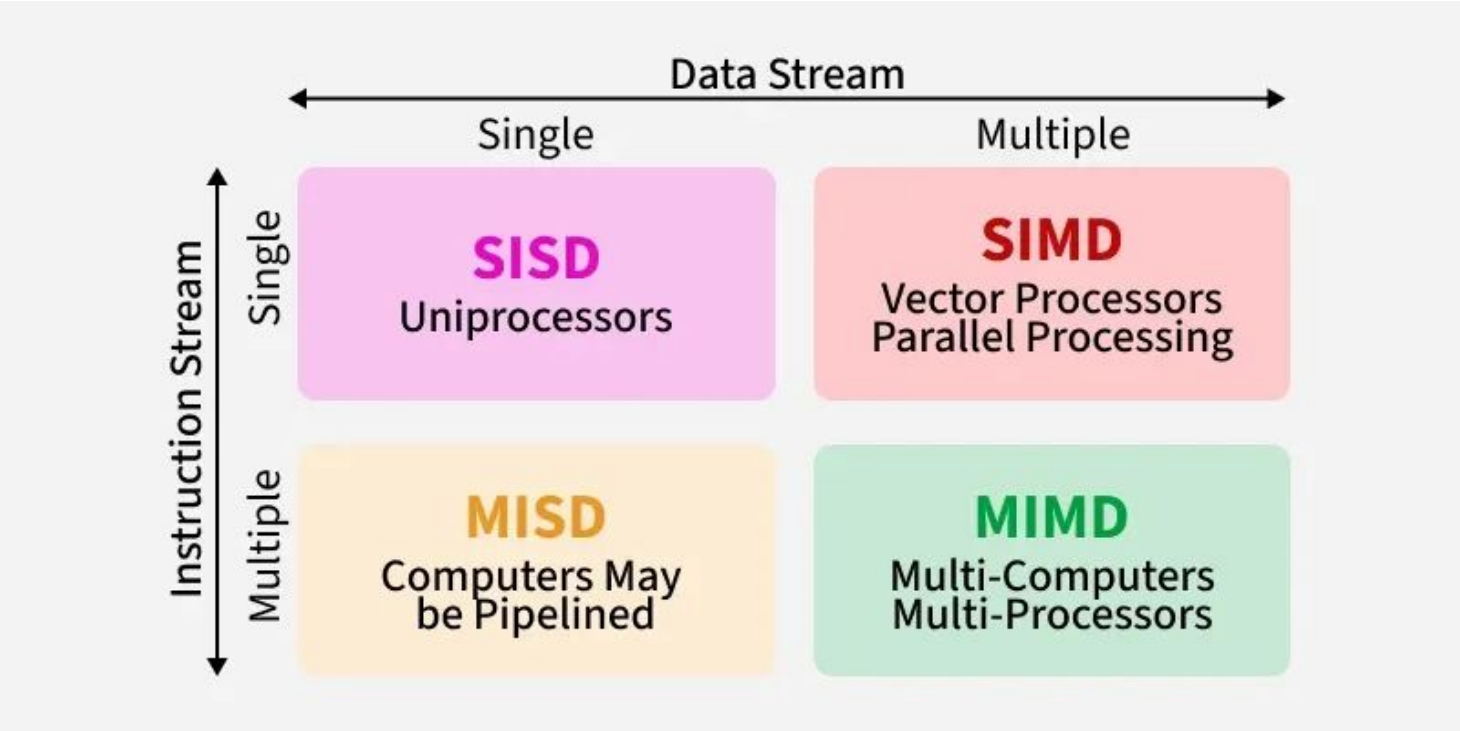
(a) System with discrete GPU



(b) Integrated CPU and GPU

Demo time!

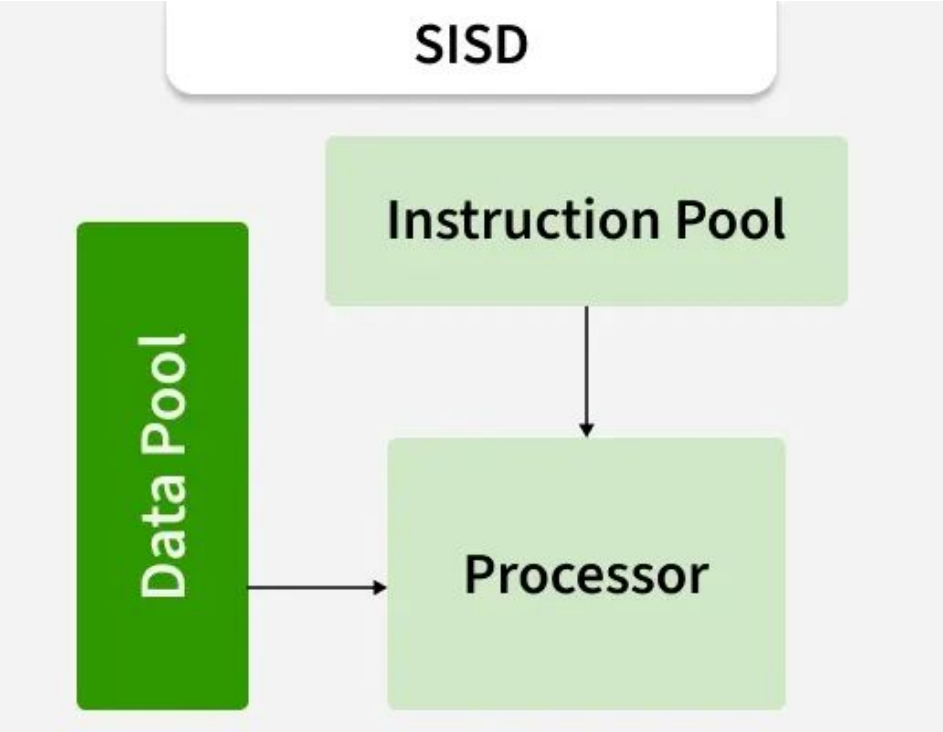
Flynn's Taxonomy



[10]

Single-Instruction, Single Data (SISD)

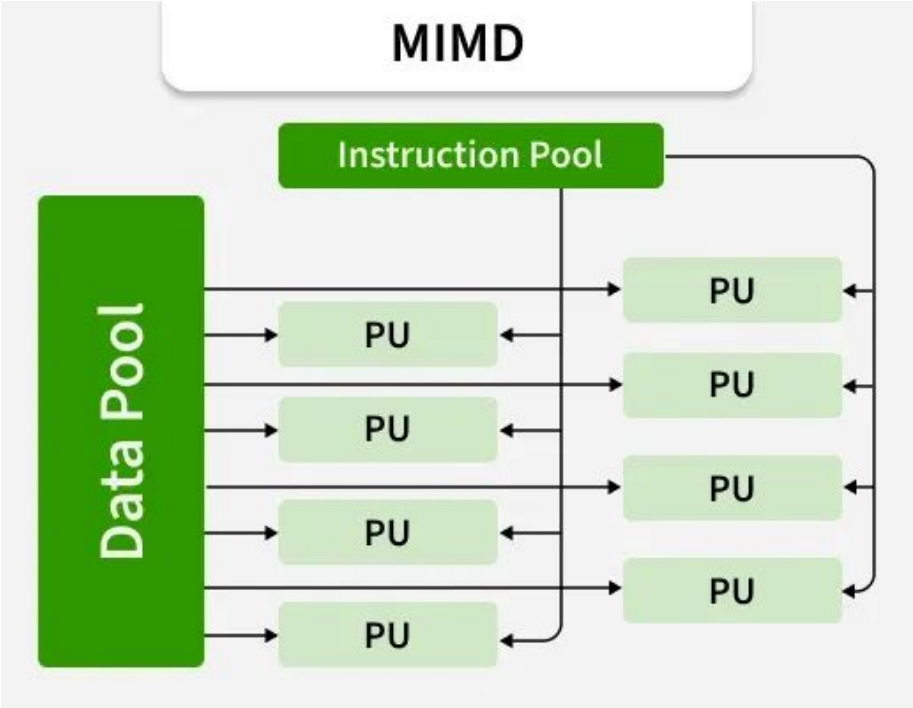
- One instruction operates on a single data stream



[10]

Multiple-instruction, multiple data (MIMD)

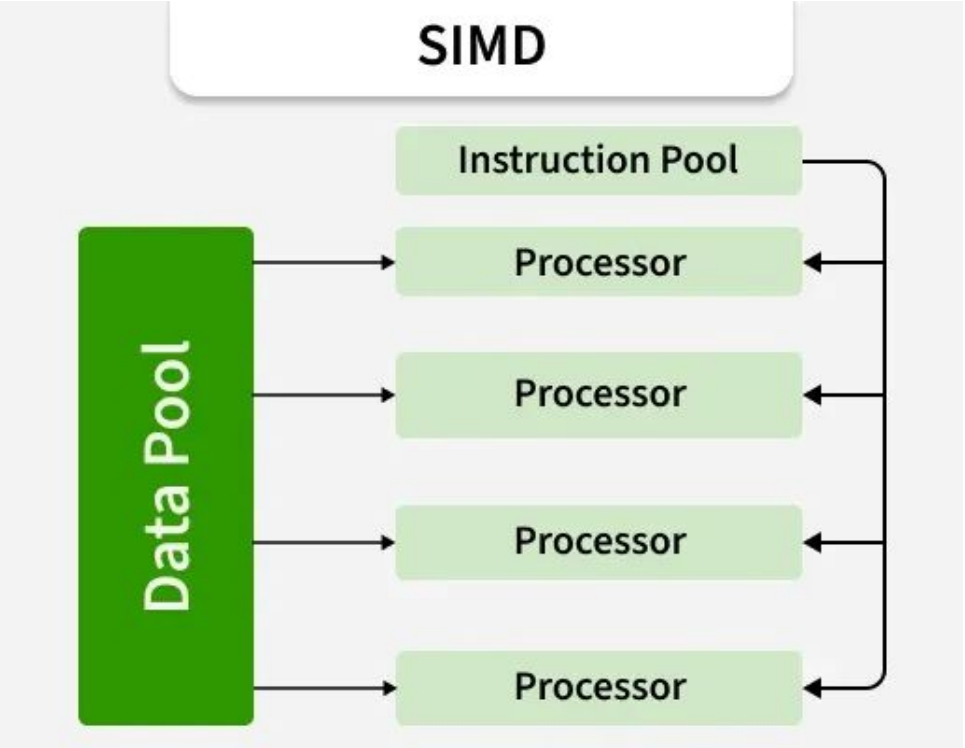
- Multiple cores that could be executing different instructions on different pieces of data
- All processors are connected via “globally available” memory



[10]

Single-Instruction, Multiple-Data (SIMD)

- A multiprocessor machine that executes the same instruction on all the processors, but operates on different data streams



[10]

Programming Model

- **SIMT (Single-instruction, multiple thread) execution model**
  - All threads share same program
- Developer can write scalar programs which are executed on the vector hardware
- Give abstraction of running thousands of independent threads in parallel

```
__global__ void saxpy(int n, float a, float *x, float *y)
{
   int i = blockIdx.x*blockDim.x + threadIdx.x;
   if(i<n)
     y[i] = a*x[i] + y[i];
}
int main() {
  float *h_x, *h_y;
  int n;
  // omitted: allocate CPU memory for h_x and h_y and initialize contents
  float *d_x, *d_y;
  int nblocks = (n + 255) / 256;
  cudaMalloc( &d_x, n * sizeof(float) );
  cudaMalloc( &d_y, n * sizeof(float) );
  cudaMemcpy( d_x, h_x, n * sizeof(float), cudaMemcpyHostToDevice );
  cudaMemcpy( d_y, h_y, n * sizeof(float), cudaMemcpyHostToDevice );
  saxpy<<<nblocks, 256>>>(n, 2.0, d_x, d_y);
  cudaMemcpy( h_x, d_x, n * sizeof(float), cudaMemcpyDeviceToHost );
  // omitted: use h_y on CPU, free memory pointed to by h_x, h_y, d_x, and d_y
}
```

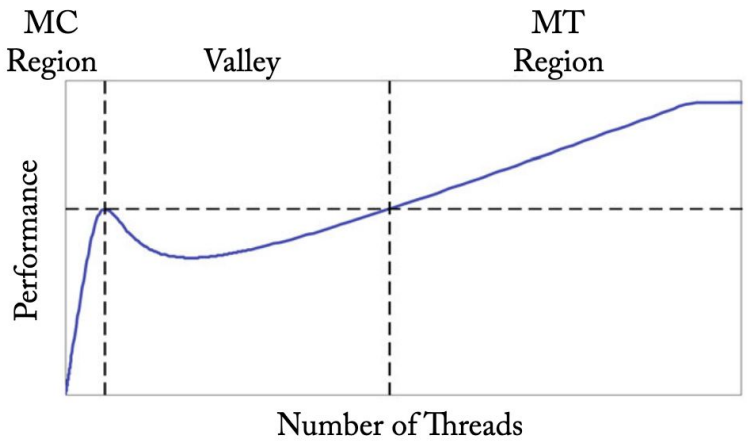
Kernel Program (GPU)

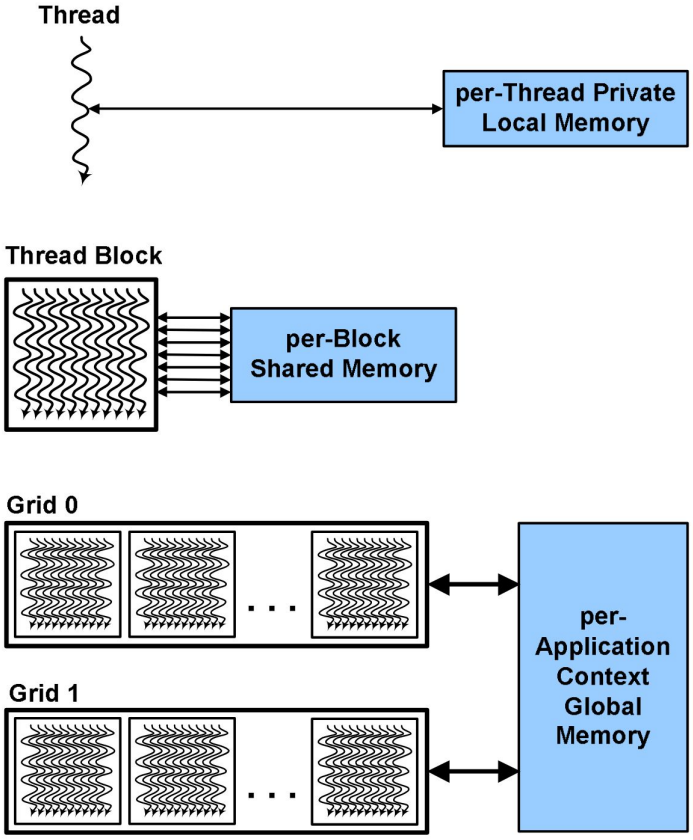
Host (CPU)

Kernel Launch

Programming Model

- Kernels are composed thousands of threads
  - Threads are grouped into warps, which operate in lock-step
  - Warps are grouped into blocks, which can sync/communicate via a shared memory
  - Blocks make up the grid e.g the kernel

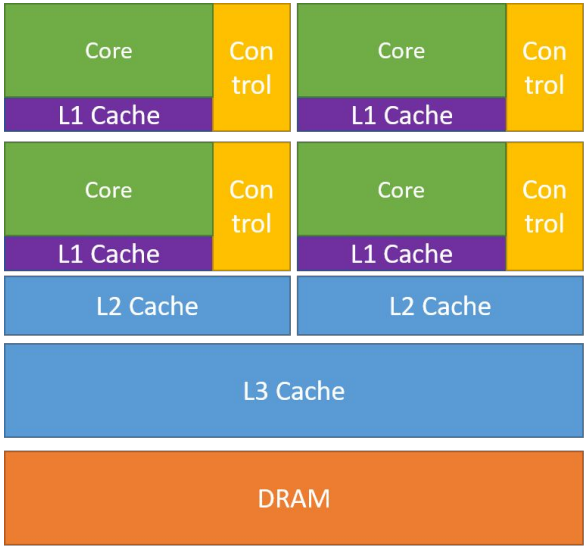




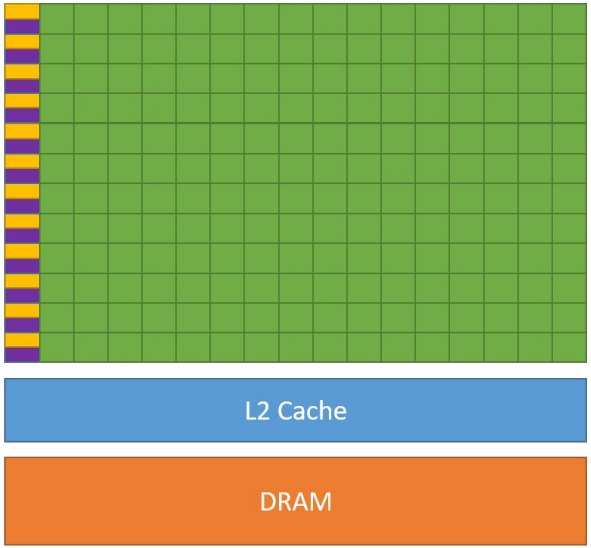
CUDA Hierarchy of threads, blocks, and grids, with corresponding per-thread private, per-block shared, and per-application global memory spaces.

GPU Architecture Overview

- Throughput optimized
- Executes thousands of threads in parallel
- More area dedicated for arithmetic logic than control



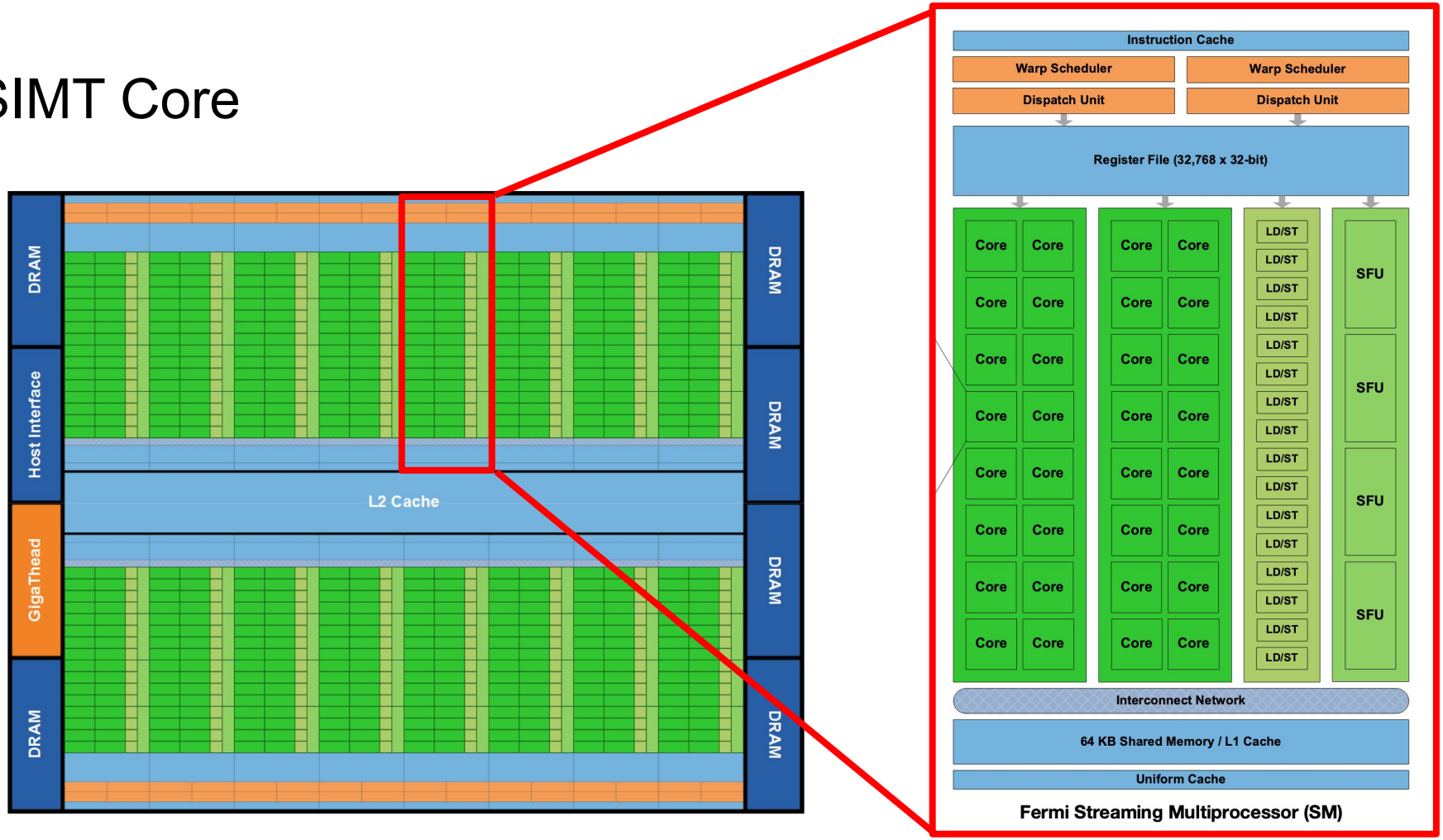
CPU



GPU

[5]

SIMT Core


DRAM
Host Interface
GigaThead
DRAM
L2 Cache
DRAM
DRAM
DRAM
DRAM
Instruction Cache
Warp Scheduler
Warp Scheduler
Dispatch Unit
Dispatch Unit
Register File (32,768 x 32-bit)
Core
LD/ST
SFU
Interconnect Network
64 KB Shared Memory / L1 Cache
Uniform Cache
Fermi Streaming Multiprocessor (SM)

[1]

SIMT Microarchitecture

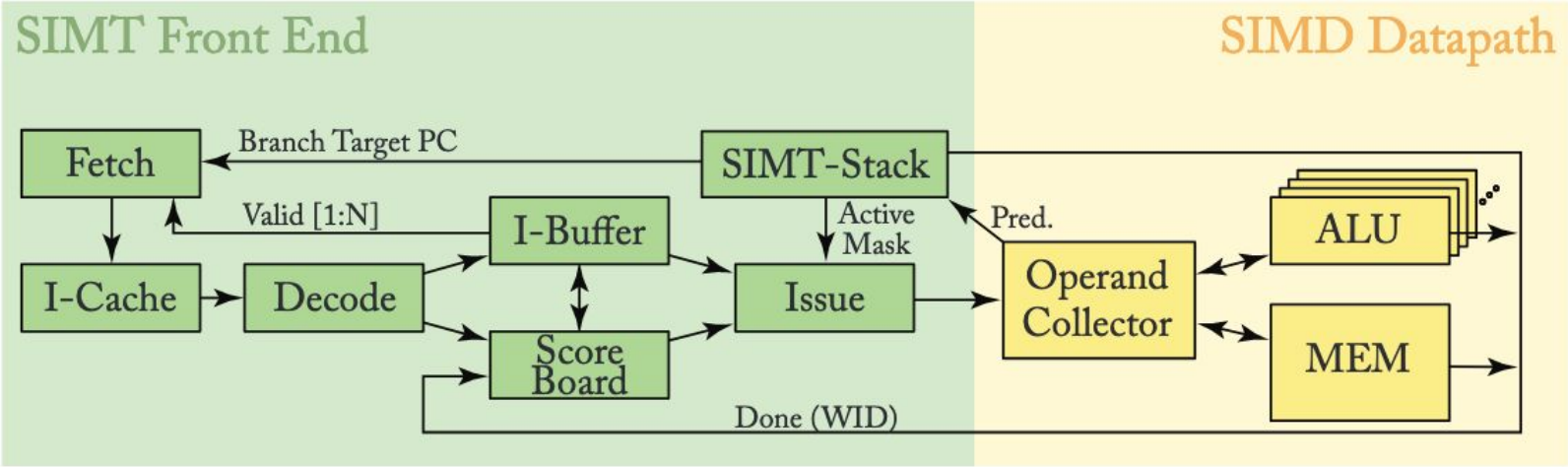
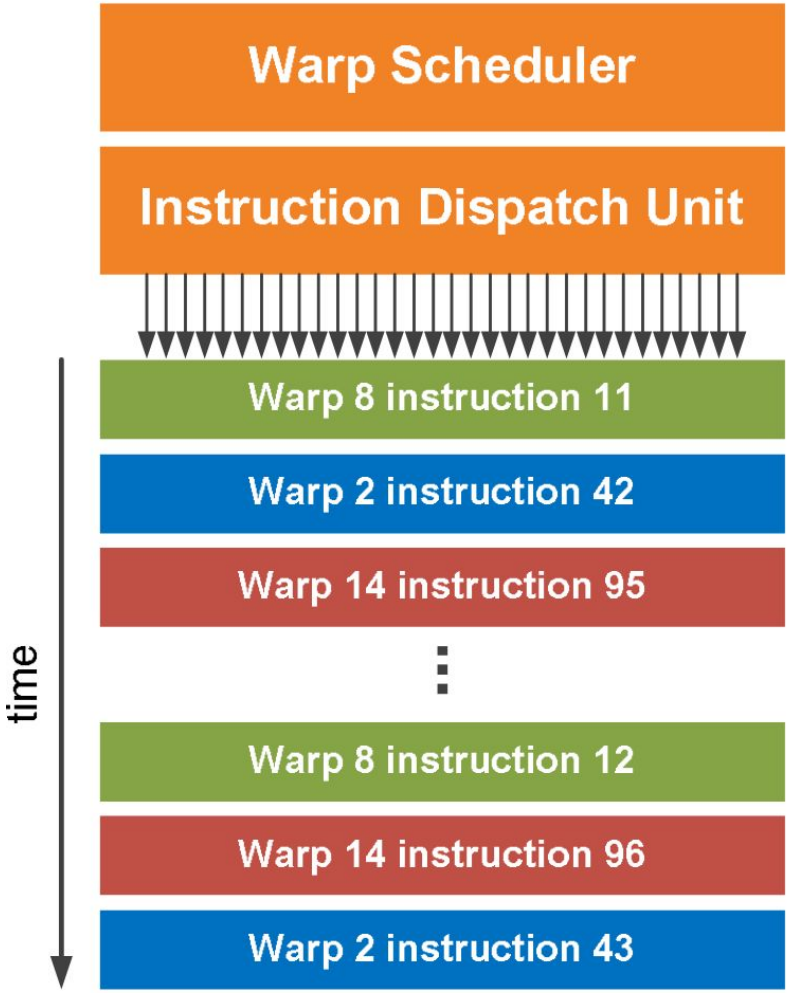


Figure 3.1: Microarchitecture of a generic GPGPU core.

[6]

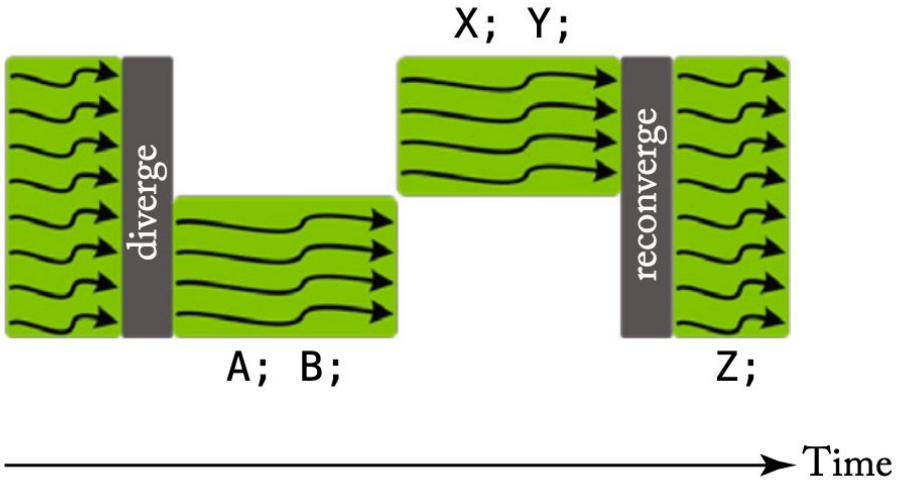
SIMT Microarchitecture



[1]

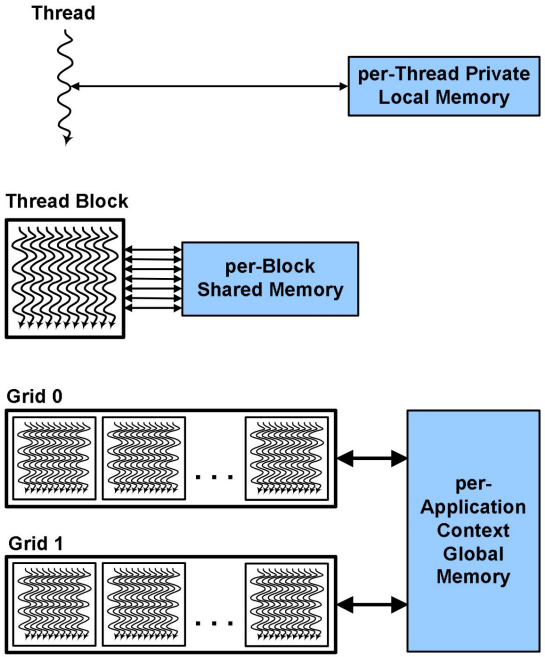
SIMT Execution Model

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```

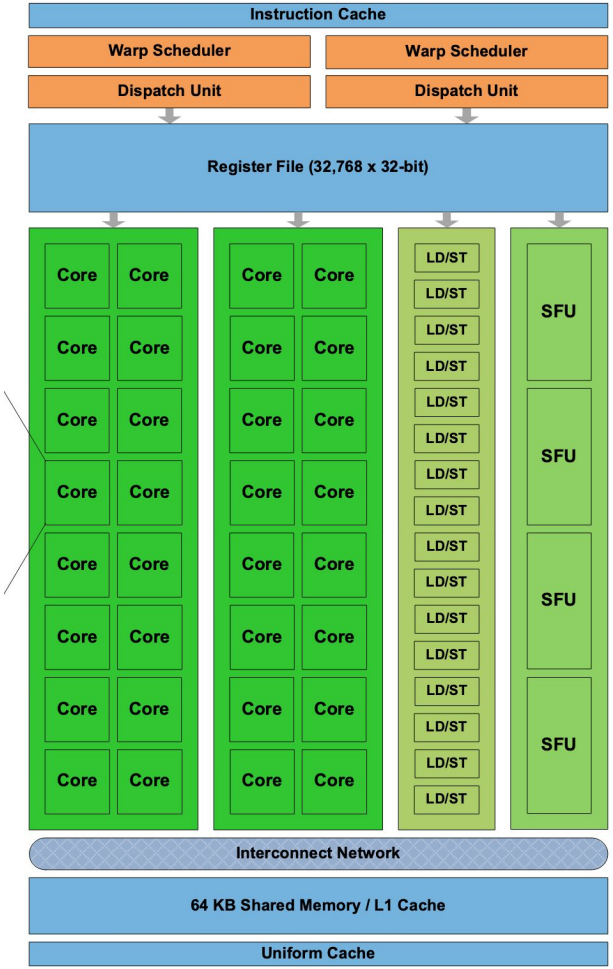


[8]

Memory System



CUDA Hierarchy of threads, blocks, and grids, with corresponding per-thread private, per-block shared, and per-application global memory spaces.



Fermi Streaming Multiprocessor (SM)

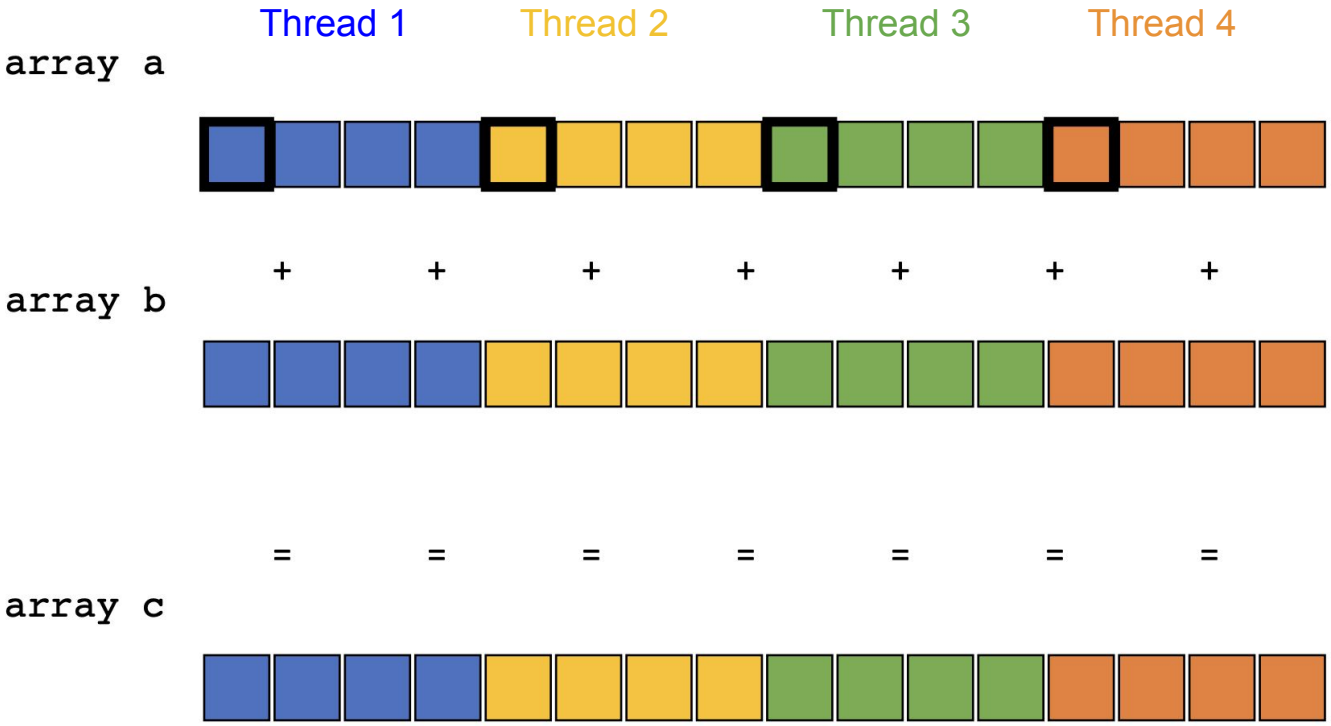
[5]

Memory Coalescing

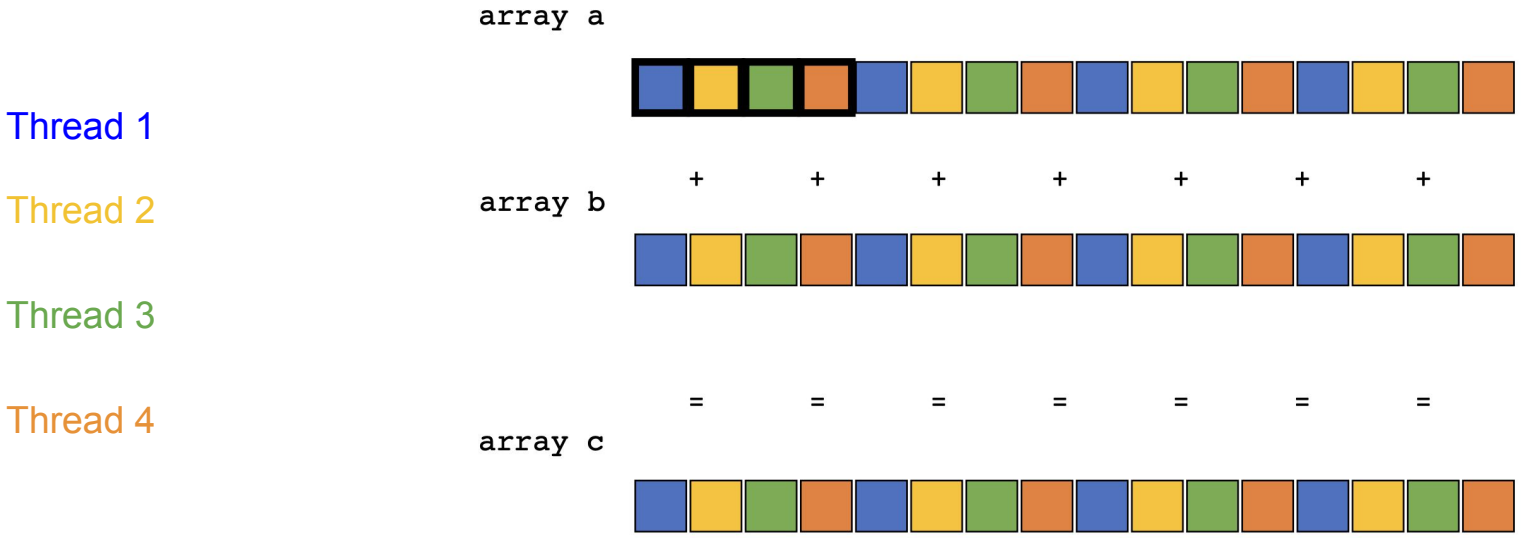
```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
   int chunk_size = size/blockDim.x;
   int start = chunk_size * threadIdx.x;
   int end = start + end;
   for (int i = start; i < end; i++) {
     d_a[i] = d_b[i] + d_c[i];
  }
}
```

- “Chunking”
  - Workload is statically partitioned between threads
  - Each thread is given a chunk of data to process

- Can result in uncoalesced memory accesses!
  - Each thread generates a memory request



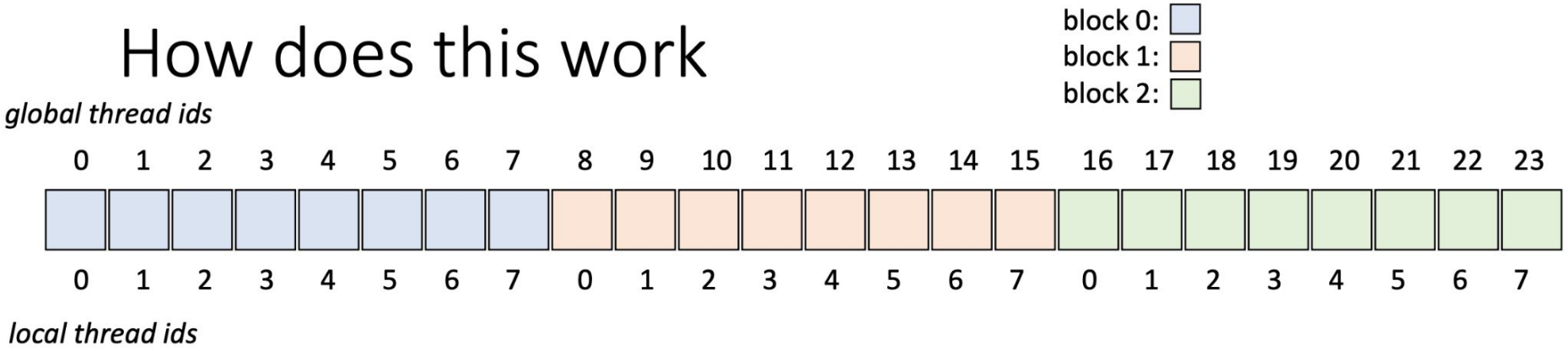
```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
  for (int i = threadIdx.x; i < size; i+=blockDim.x) {
     d_a[i] = d_b[i] + d_c[i];
  }
}
```



- “Interleaved”: memory accesses get **coalesced**
  - **All threads saturated by single memory request**

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  d_a[i] = d_b[i] + d_c[i];
}
```

How does this work



Advanced Kernel Programming

- **GPU programming is hard**
  - Memory coalescing, bank conflicts, thread divergence, occupancy, kernel fusion, thread block size, workload balancing, synchronization
  - GPU libraries implement all of these optimizations
    - cuBLAS, cuDNN, cuFFT, etc
    - Optimized per device
  - Non trivial parallel algorithms (“humblingly parallel”) are often extremely complex to implement on GPU
    - Prefix scan, graph algorithms, dynamic task parallelism

CPU vs GPU

	CPU	**GPU**
Function	General-purpose processing for sequential tasks	Specialized for highly parallel processing and data-intensive tasks
Architecture	A few powerful cores	Thousands of smaller, simpler cores
Core count	4-64	Thousands
Memory bandwidth	Lower, optimized for latency	Higher, optimized for throughput
Clock speed	Higher (up to ~5 GHz)	Lower (~1-2 GHz)

What applications are GPUs good at?

- Applications where you have large amounts of data that need to have the same instruction run on the data with little to no branches
  - Graphics and video processing
  - Machine learning and AI
  - Data science and scientific computing

Now, back to our demo!

Summary/Key Takeaways

- **Massive Parallelism**
  - GPUs have thousands of cores designed for executing many threads simultaneously.
- **SIMT (Single Instruction, Multiple Threads) Model:**
  - Each core executes the same instruction across multiple threads, reducing the overhead of instruction dispatch.
  - This enables efficient execution of tasks with regular patterns, like matrix operations.
- **Memory Hierarchy & Bandwidth:**
  - Optimized memory access patterns, such as coalesced memory accesses, help minimize latency.
- **Latency Hiding Through Thread Scheduling:**
  - GPUs hide memory access latency by rapidly switching between threads.
  - While some threads wait for memory, others can continue executing, keeping compute units busy.

Questions?

References

[1] https://www.nvidia.com/content/PDF/fermi_white_papers/NVIDIA_Fermi_Compute_Architecture_Whitepaper.pdf

[2] https://dl.acm.org/doi/10.1145/882262.882363

[3] https://leeyngdo.github.io/blog/computer-graphics/2024-02-29-graphics-pipeline/

[4] https://www.techpowerup.com/gpu-specs/geforce3.c738

[5] https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[6] General-Purpose Graphics Processor Architectures | SpringerLink

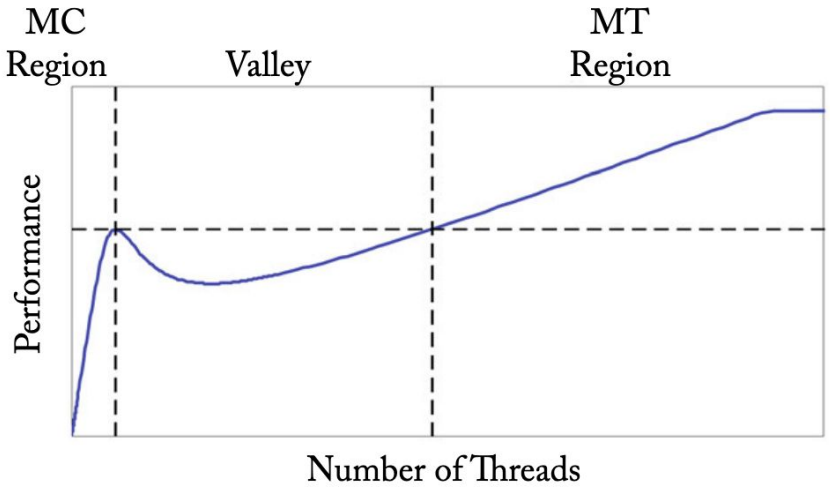
[7] https://ppl.stanford.edu/papers/ppopp070a-hong.pdf\

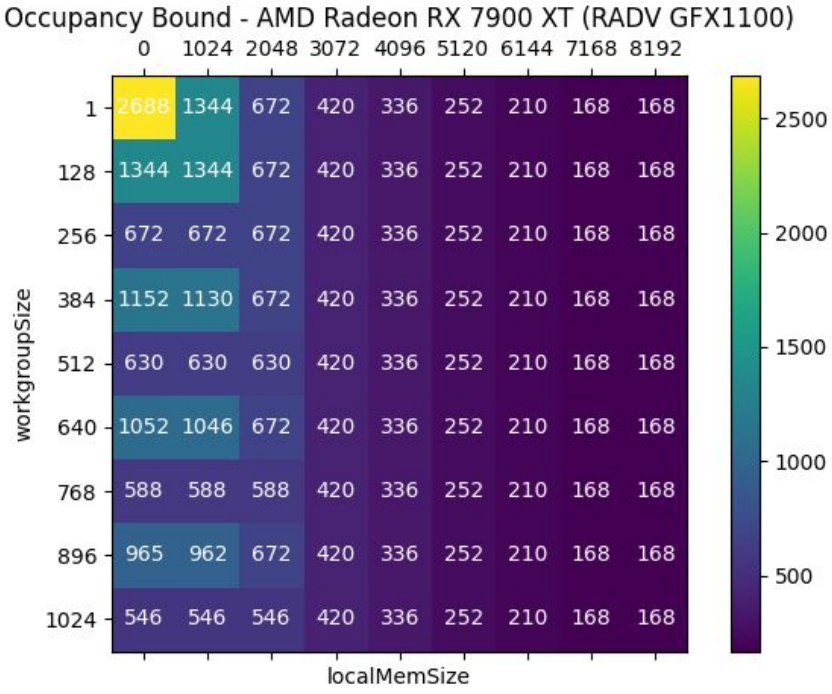
[8] https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

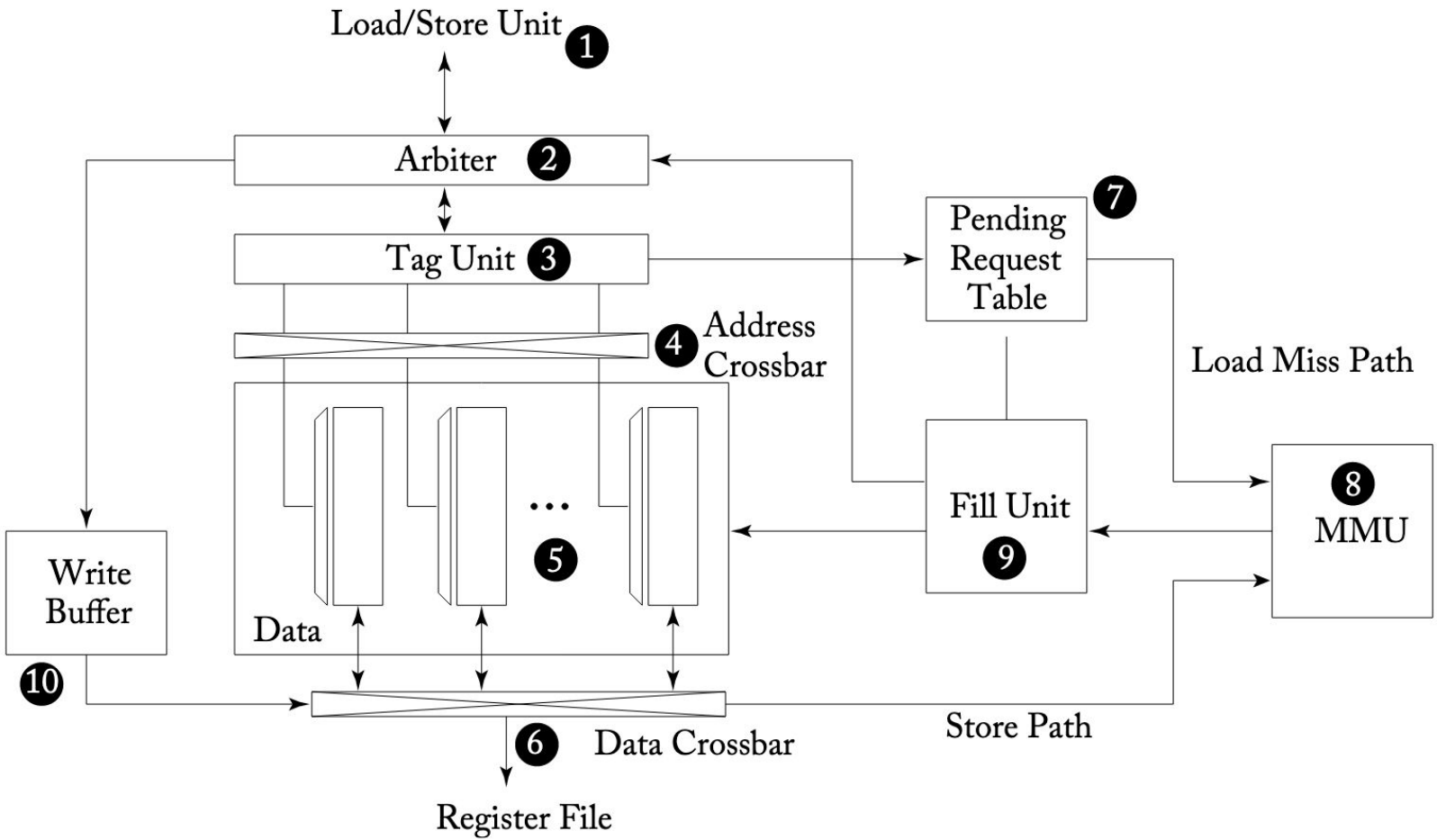
[9] https://sorensenucsc.github.io/CSE113-wi2022/lectures/CSE113March9_wi2022.pdf

[10] https://www.geeksforgeeks.org/computer-organization-architecture/computer-architecture-flynns-taxonomy/

Warp Scheduling







[6]

Figure 4.1: Unified L1 data cache and shared memory [Minkin et al., 2012].

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
__syncwarp()
```

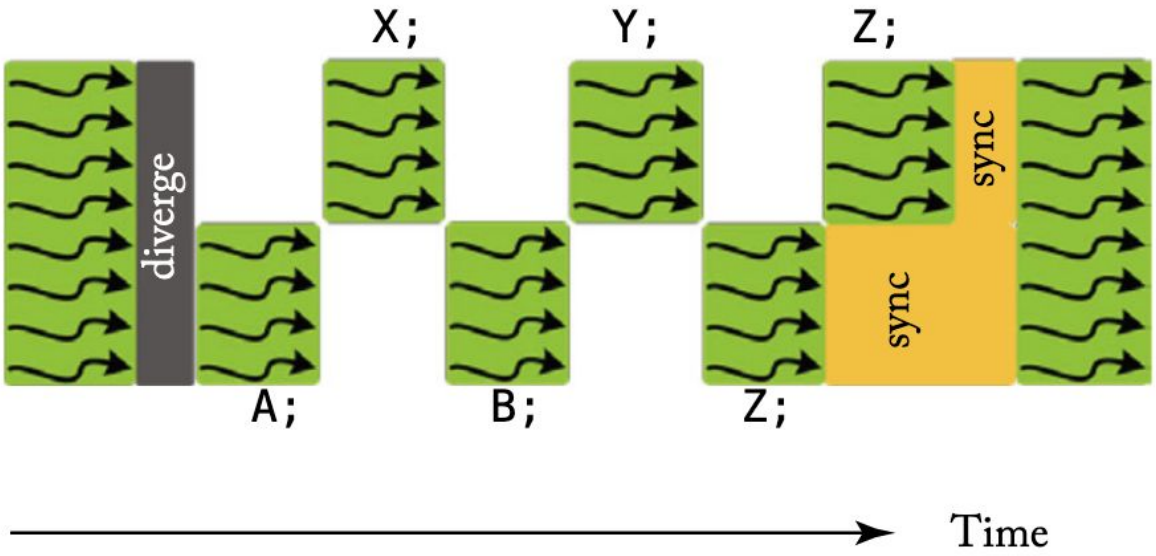


Figure 3.10: Example showing behavior of Volta reconvergence (based on Figure 23 from Nvidia [2017]).

[6]

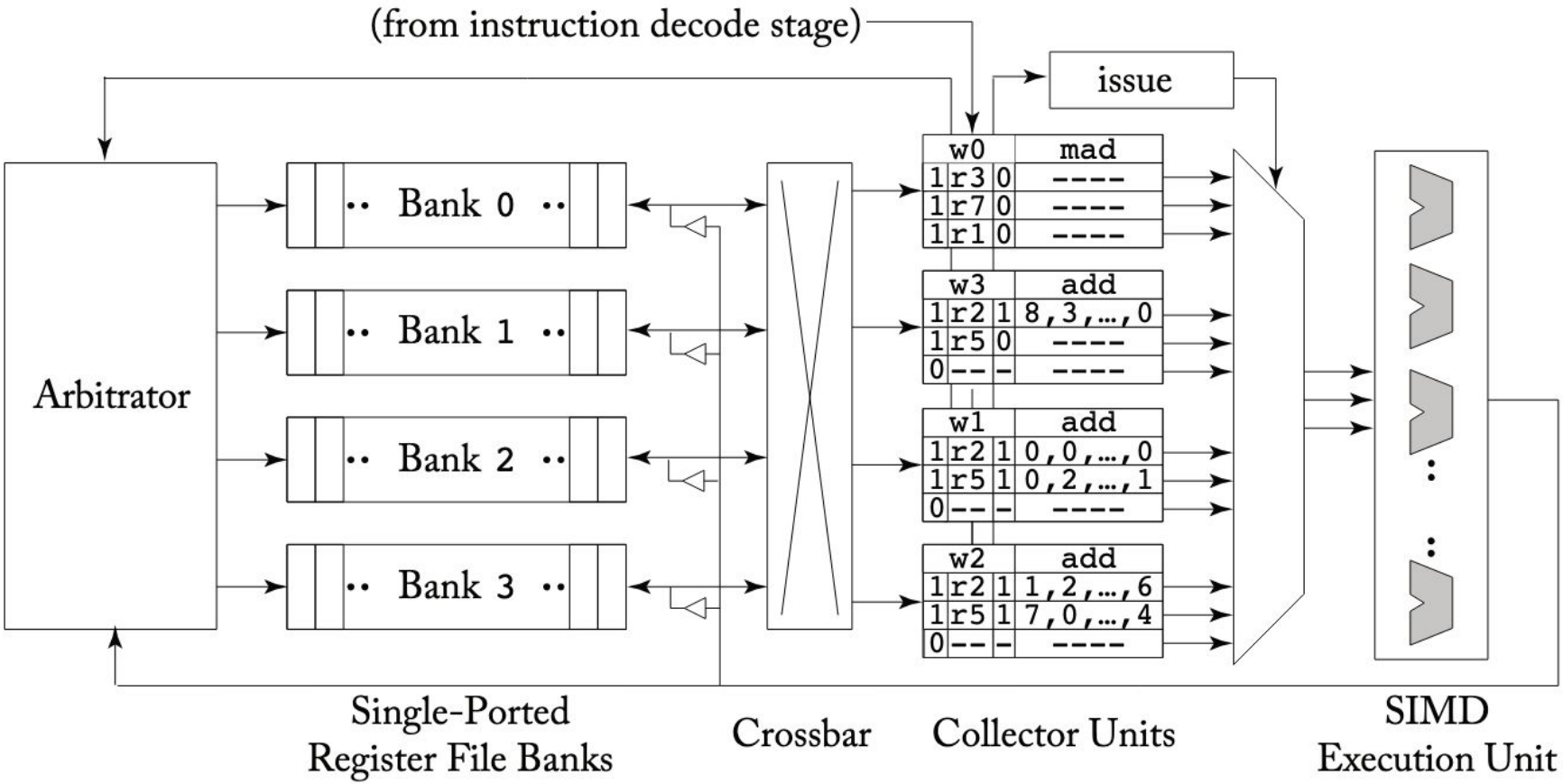


Figure 3.15: Operand collector microarchitecture (based on Figure 6 from Tor M. Aamodt et al.). [6]